

A/B Testing for Business Analytics

Design, measurement, statistics, code, and business decisions

Armando Vieira

Business Analytics

May 9, 2026

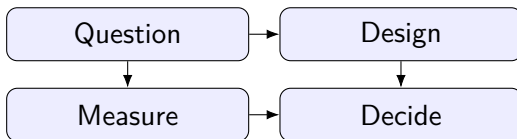
Learning goals

By the end, students should be able to:

- Translate a business question into a testable hypothesis.
- Design a clean randomized experiment.
- Choose metrics, sample size, and duration.
- Analyze results with confidence intervals and effect sizes.
- Avoid common A/B testing traps.

Core idea

A/B testing is not just statistics. It is a disciplined way to reduce business uncertainty.

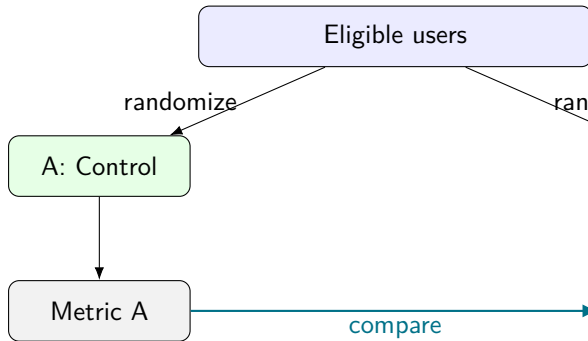


What is an A/B test?

Definition

A controlled experiment where users, customers, stores, or sessions are randomly assigned to variants.

- A: control / current experience
- B: treatment / proposed change
- Goal: estimate causal impact



Business use case: e-commerce checkout

Scenario

An online store wants to test a simplified checkout page.

Business hypothesis

Reducing checkout friction increases completed purchases without increasing refunds or support tickets. **Variants**

- A: current checkout
- B: shorter checkout, fewer fields

Metrics

- Primary: conversion rate
- Secondary: revenue per visitor
- Guardrail: refund rate, support contacts

Decision question

Is B better enough to justify rollout risk?

From business problem to hypothesis

Business question	Experimental hypothesis	Metric
Can we increase purchase completion?	A shorter checkout increases conversion.	Conversion rate
Can we reduce churn?	A proactive retention offer reduces 30-day churn.	Churn rate
Can we grow basket size?	Product bundles increase average order value.	Revenue/order
Can we improve onboarding?	Guided onboarding raises activation.	Activation rate

Good hypothesis template

For *population X*, changing *A* to *B* will improve *primary metric Y* by at least *MDE*, while keeping *guardrail Z* acceptable.

The experimental unit matters

Common units

- User/customer
- Session
- Account/company
- Store/region
- Email recipient

Rule Randomize at the level where interference is lowest and business action is made.

Example

Testing a loyalty discount? Randomize by customer, not transaction, otherwise the same person may experience both A and B.

Random assignment in Python

```
1 import pandas as pd
2 import numpy as np
3
4 customers = pd.read_csv("eligible_customers.csv")
5
6 rng = np.random.default_rng(seed=42)
7 customers["variant"] = np.where(
8     rng.random(len(customers)) < 0.50,
9     "A_control",
10    "B_treatment"
11 )
12
13 # Always persist assignment: users must stay in the same group
14 customers[["customer_id", "variant"]].to_csv(
15     "ab_assignment.csv", index=False
16 )
```

Recommendation

Use deterministic or stored assignment. Never re-randomize returning users.

Random assignment with a hash rule

Useful when assignment must be stable across systems.

```
1 import hashlib
2
3 def assign_variant(user_id, split=0.50, salt="checkout_test_2026"):
4     key = f"{salt}:{user_id}".encode("utf-8")
5     h = hashlib.md5(key).hexdigest()
6     score = int(h[:8], 16) / 16**8
7     return "B_treatment" if score < split else "A_control"
8
9 assign_variant("customer_123")
```

Why this is useful

The same user always gets the same variant, even if the assignment is computed in a different application.

Metric design: primary, secondary, guardrail

Primary metric	Secondary metrics	Guardrail metrics
Metric used for the main decision.	Explain why the primary metric changed.	Protect the business from hidden damage.
Example: conversion rate.	Example: add-to-cart rate, checkout start rate.	Example: refunds, latency, complaints.

Bad practice: declare victory on whichever metric looks best after seeing the data.

Minimum detectable effect: the business threshold

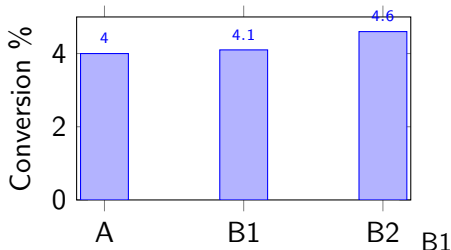
MDE

The smallest effect worth detecting and acting on.

Example

Current conversion rate is 4.0%. A change to 4.1% may be statistically detectable but operationally irrelevant.

- Effect size must matter economically.
- Include implementation cost and risk.
- Predefine before running the test.



may be too small to matter; B2 may justify rollout.

Sample size calculation for conversion rate

```
1 from statsmodels.stats.power import NormalIndPower
2 from statsmodels.stats.proportion import proportion_effectsize
3
4 baseline = 0.04          # current conversion rate
5 mde = 0.004             # +0.4 percentage points
6 alpha = 0.05            # false positive risk
7 power = 0.80            # probability of detecting MDE
8
9 analysis = NormalIndPower()
10 effect_size = proportion_effectsize(baseline, baseline + mde)
11
12 n_per_group = analysis.solve_power(
13     effect_size=effect_size,
14     alpha=alpha,
15     power=power,
16     ratio=1,
17     alternative="two-sided"
18 )
19 print(round(n_per_group))
```

Interpretation

Low baseline rates and small MDEs require large samples

How long should the test run?

Duration depends on

- Required sample size
- Daily eligible traffic
- Weekly seasonality
- Delayed outcomes
- Data pipeline latency

Practical rule

Run for at least one full business cycle, often one or two full weeks, unless the product has a different natural cycle.

Warning

Do not stop just because the p-value became attractive today.

Collecting experiment data with SQL

```
1 SELECT
2     a.variant,
3     COUNT(DISTINCT a.customer_id) AS users,
4     COUNT(DISTINCT o.order_id) AS orders,
5     SUM(o.revenue) AS revenue,
6     COUNT(DISTINCT o.order_id) * 1.0 /
7         COUNT(DISTINCT a.customer_id) AS conversion_rate,
8     SUM(o.revenue) * 1.0 /
9         COUNT(DISTINCT a.customer_id) AS revenue_per_user
10 FROM ab_assignment a
11 LEFT JOIN orders o
12     ON a.customer_id = o.customer_id
13     AND o.order_date BETWEEN '2026-03-01' AND '2026-03-14'
14 GROUP BY a.variant;
```

Important

Use the assignment table as the base population. Do not analyze only users who converted.

Analyzing conversion: two-proportion z-test

```
1 from statsmodels.stats.proportion import proportions_ztest,  
   confint_proportions_2indep  
2  
3 # Example results  
4 conversions = [420, 470]    # A, B  
5 visitors = [10000, 10050]  
6  
7 z_stat, p_value = proportions_ztest(conversions, visitors)  
8 ci_low, ci_high = confint_proportions_2indep(  
9     count1=conversions[1], nobs1=visitors[1],  
10    count2=conversions[0], nobs2=visitors[0],  
11    method="wald"  
12 )  
13  
14 lift = conversions[1]/visitors[1] - conversions[0]/visitors[0]  
15 print(z_stat, p_value, lift, (ci_low, ci_high))
```

Report

Report lift and confidence interval, not only p-value.

Results table: what students should present

Variant	Users	Conversions	Conversion	Revenue/user
A control	10,000	420	4.20%	\$2.38
B treatment	10,050	470	4.68%	\$2.61
Difference	–	–	+0.48 pp	+\$0.23
95% CI	–	–	[+0.05, +0.91] pp	–

Business interpretation

The treatment appears to improve conversion. Now check guardrails, implementation cost, and whether the effect is consistent in important segments.

Analyzing continuous metrics: revenue per user

Revenue data is often skewed. Start simple, then add robustness.

```
1 import pandas as pd
2 from scipy import stats
3
4 # df has columns: user_id, variant, revenue
5 A = df.loc[df.variant == "A_control", "revenue"]
6 B = df.loc[df.variant == "B_treatment", "revenue"]
7
8 # Welch t-test: does not assume equal variance
9 stat, p = stats.ttest_ind(B, A, equal_var=False)
10
11 lift = B.mean() - A.mean()
12 print("mean A", A.mean())
13 print("mean B", B.mean())
14 print("lift", lift, "p", p)
```

Recommendation

For revenue, also use bootstrap confidence intervals or winsorization sensitivity checks.

Bootstrap confidence interval

```
1 import numpy as np
2
3 rng = np.random.default_rng(42)
4
5 def bootstrap_diff(A, B, n_boot=5000):
6     diffs = []
7     A = np.array(A); B = np.array(B)
8     for _ in range(n_boot):
9         A_sample = rng.choice(A, size=len(A), replace=True)
10        B_sample = rng.choice(B, size=len(B), replace=True)
11        diffs.append(B_sample.mean() - A_sample.mean())
12    return np.percentile(diffs, [2.5, 50, 97.5])
13
14 ci = bootstrap_diff(A, B)
15 print("median effect and 95% CI:", ci)
```

Why bootstrap?

It is useful when metric distributions are non-normal, skewed, or hard to model analytically.

Technique 1: stratified randomization

Use stratification when known groups have very different baseline behavior. **Examples**

- New vs returning customers
- Country or region
- Mobile vs desktop
- High-value vs low-value customers



Benefit

Better balance and often more statistical power.

Stratified assignment in Python

```
1 import numpy as np
2
3 rng = np.random.default_rng(42)
4
5 def stratified_assign(group):
6     random_scores = rng.random(len(group))
7     group = group.copy()
8     group["variant"] = np.where(random_scores < 0.5, "A", "B")
9     return group
10
11 customers = customers.groupby(
12     ["country", "device_type"],
13     group_keys=False
14 ).apply(stratified_assign)
```

Note

Stratification must be based on variables known before treatment exposure.

Technique 2: CUPED for variance reduction

Idea

Use a pre-experiment covariate to remove predictable noise from the outcome.

$$Y_i^{adj} = Y_i - \theta(X_i - \bar{X}), \quad \theta = \frac{\text{Cov}(Y, X)}{\text{Var}(X)}$$

Example

Use customer spend in the previous month to adjust current experiment revenue.

When useful

High correlation between past and current behavior.

CUPED code example

```
1 import numpy as np
2
3 # y = experiment revenue per user
4 # x = pre-period revenue per user
5 x = df["pre_revenue"].values
6 y = df["experiment_revenue"].values
7
8 theta = np.cov(y, x, ddof=1)[0, 1] / np.var(x, ddof=1)
9 df["revenue_cuped"] = y - theta * (x - x.mean())
10
11 A = df.loc[df.variant == "A", "revenue_cuped"]
12 B = df.loc[df.variant == "B", "revenue_cuped"]
13 print("Adjusted lift:", B.mean() - A.mean())
```

Caution

Do not use variables affected by the treatment as covariates.

Technique 3: factorial experiments

Test two or more factors at once.

- Button color: blue vs green
- Message: rational vs emotional
- Discount: 0% vs 10%

Cell	Message	Discount
1	Rational	0%
2	Rational	10%
3	Emotional	0%
4	Emotional	10%

Value

Can estimate interaction effects, not only isolated changes.

Technique 4: A/A tests

What is an A/A test?

Randomize users into two groups that both receive the same experience.

Purpose

- Validate randomization
- Detect tracking bugs
- Estimate natural metric variance
- Check false positive behavior

Expected result

No systematic difference between groups.
Occasional small differences are normal.

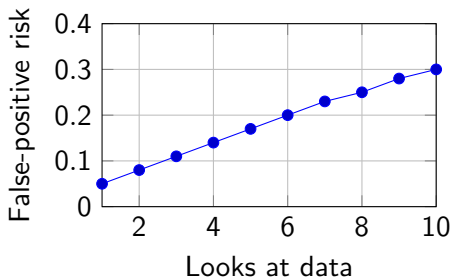
Business use

Run A/A before launching a new experimentation platform.

Technique 5: sequential testing and peeking

Repeatedly checking a normal p-value increases false positives.

- Predefine stopping rules.
- Use sequential methods if monitoring continuously.
- Separate monitoring dashboards from decision rules.



Illustrative: more unplanned looks can inflate error.

Design traps

- Unclear hypothesis
- Wrong unit of randomization
- Sample ratio mismatch
- Users exposed to both variants
- Running during abnormal events

Analysis traps

- Peeking and early stopping
- Ignoring guardrails
- Multiple testing without correction
- Overinterpreting segments
- Confusing correlation with causality outside the test

Sample ratio mismatch check

If the planned split is 50/50, large imbalance may indicate implementation problems.

```
1 from scipy.stats import chisquare
2
3 observed = [10000, 9500]          # A, B users
4 expected = [9750, 9750]          # expected 50/50 split
5
6 chi2, p = chisquare(f_obs=observed, f_exp=expected)
7 print("SRM p-value:", p)
8
9 if p < 0.001:
10     print("Investigate assignment or logging before analyzing effects")
```

Recommendation

Treat serious sample ratio mismatch as a data quality incident.

Problem

If you test many metrics and many segments, something will look significant by chance.

Better approach

- Predefine key segments.
- Keep one primary metric.
- Use corrections for many comparisons.
- Treat exploratory findings as new hypotheses.

Case	Method
Few planned tests	Bonferroni/Holm
Many exploratory tests	FDR control
Segment discovery	Follow-up experiment

Logistic regression for adjusted analysis

Useful for binary outcomes and covariates.

```
1 import statsmodels.formula.api as smf
2
3 # converted is 0/1; treatment is 1 for B and 0 for A
4 model = smf.logit(
5     "converted ~ treatment + C(device) + C(country) + prior_orders",
6     data=df
7 ).fit()
8
9 print(model.summary())
10 print("Treatment odds ratio:", np.exp(model.params["treatment"]))
```

Use carefully

Randomization gives causal identification. Regression mainly improves precision or adjusts small imbalances.

Bayesian A/B testing: decision-oriented view

Instead of only asking “is the effect non-zero?”,
Bayesian analysis can ask:

- Probability B is better than A.
- Probability lift exceeds the business threshold.
- Expected value of rolling out B.

$$\theta_A \sim \text{Beta}(1, 1), \quad \theta_B \sim \text{Beta}(1, 1)$$

$$\theta_A | \text{data} \sim \text{Beta}(1 + x_A, 1 + n_A - x_A)$$

$$\theta_B | \text{data} \sim \text{Beta}(1 + x_B, 1 + n_B - x_B)$$

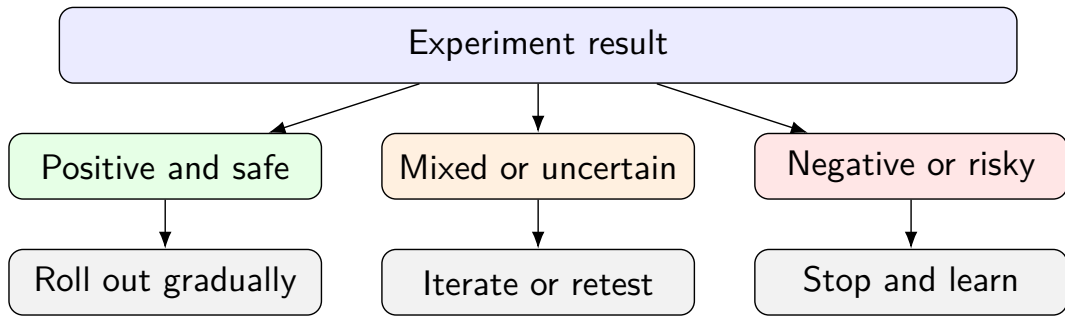
Recommendation

Bayesian methods are useful, but still require clean design and stopping discipline.

Bayesian simulation for conversion

```
1 import numpy as np
2
3 rng = np.random.default_rng(42)
4 x_A, n_A = 420, 10000
5 x_B, n_B = 470, 10050
6
7 samples_A = rng.beta(1 + x_A, 1 + n_A - x_A, 100_000)
8 samples_B = rng.beta(1 + x_B, 1 + n_B - x_B, 100_000)
9
10 lift = samples_B - samples_A
11 print("P(B > A):", np.mean(lift > 0))
12 print("P(lift > 0.003):", np.mean(lift > 0.003))
13 print("95% credible interval:", np.percentile(lift, [2.5, 97.5]))
```

Business decision framework



Business rule

A statistically significant result is not automatically a business decision.

Rollout strategy after a winning test

- Start with a gradual rollout: 10%, 25%, 50%, 100%.
- Continue monitoring guardrails.
- Check operational capacity: support, inventory, fraud, infrastructure.
- Document the decision and expected impact.
- Keep a holdout if long-term effects matter.

Example

A discount that improves conversion may reduce margin or train customers to wait for promotions.

Design

- Hypothesis written
- Population defined
- Unit of randomization chosen
- Variants implemented
- Sample size estimated

Measurement

- Primary metric selected
- Guardrails selected
- Tracking validated
- Assignment persisted
- Analysis plan written

Post-test checklist

- Did the experiment run for the planned duration?
- Was there sample ratio mismatch?
- Were there tracking or data pipeline incidents?
- What is the effect size and uncertainty interval?
- Are guardrails acceptable?
- Is the result robust across key segments?
- What is the expected business value?
- What should be rolled out, retested, or abandoned?

Task

Design an A/B test for a subscription business that wants to reduce churn.

Students should define:

- ① Business hypothesis
- ② Population and randomization unit
- ③ Control and treatment
- ④ Primary, secondary, and guardrail metrics
- ⑤ MDE and sample size logic
- ⑥ Analysis plan
- ⑦ Final rollout decision rule

A/B testing is a business discipline supported by statistics.

A strong experiment answers three questions

- 1 Did the treatment change the metric?
- 2 Is the effect large and reliable enough?
- 3 Should the business act on it now?

Armando Vieira

Appendix: useful Python packages

Task	Package
Data manipulation	pandas, numpy
Classical tests	scipy, statsmodels
Power analysis	statsmodels.stats.power
Regression	statsmodels, scikit-learn
Bayesian modeling	scipy, PyMC
Visualization	matplotlib, plotly
Experiment tracking	database tables, analytics platforms